DBMS – SQL QUERY IMPLEMENTATION FOR E-COMMERCE DATABASE

Project Title: Etsy Marketplace Database System

Skill Task: DBMS

Week: 7

Objectives

1. Perform SELECT, WHERE, ORDER BY, and DISTINCT queries.
2. Search products based on price, category, and availability.
3. Retrieve customer and product information.
4. Apply filtering conditions.
5. Generate basic business reports.

Tables Used

- Category (Category_ID, Category_Name)
- Product (Product_ID, Product_Name, Category_ID, Price, Stock_Quantity)
- Customer (Customer_ID, Customer_Name, Email, Contact_No, Address, Registration_Date)
- Orders (Order_ID, Customer_ID, Order_Date, Total_Amount)
- Order_Item (Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)

1. Basic SQL Queries

1.1 SELECT Query

The SELECT statement is used to retrieve data from a table.

Display all products

```
SELECT * FROM Product;
```

*Output: (paste SQL*Plus screenshot here)*

Display selected product details

```
SELECT Product_ID, Product_Name, Price
FROM Product;
```

*Output: (paste SQL*Plus screenshot here)*

1.2 WHERE Clause

The WHERE clause is used to filter records based on a condition.

Find products with price less than 1000

```
SELECT *
FROM Product
WHERE Price < 1000;
```

*Output: (paste SQL*Plus screenshot here)*

Find products with price greater than 100

```
SELECT *
FROM Product
WHERE Price > 100;
```

*Output: (paste SQL*Plus screenshot here)*

1.3 ORDER BY

The ORDER BY clause is used to sort the query results.

Products from lowest to highest price

```
SELECT Product_ID, Product_Name, Price
FROM Product
ORDER BY Price ASC;
```

*Output: (paste SQL*Plus screenshot here)*

Products from highest to lowest price

```
SELECT Product_ID, Product_Name, Price
FROM Product
ORDER BY Price DESC;
```

*Output: (paste SQL*Plus screenshot here)*

1.4 DISTINCT

The DISTINCT keyword is used to display unique values.

Display unique categories

```
SELECT DISTINCT Category_ID
FROM Product
ORDER BY Category_ID;
```

This displays each category ID that has at least one product, without repeating it.

*Output: (paste SQL*Plus screenshot here)*

2. Search Products

2.1 Search Products Based on Price

Products priced between ₹100 and ₹500

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price BETWEEN 100 AND 500;
```

This displays products whose price is between ₹100 and ₹500.

*Output: (paste SQL*Plus screenshot here)*

2.2 Search Products Based on Category

Products in category 1

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE Category_ID = 1;
```

This displays products belonging to category 1.

*Output: (paste SQL*Plus screenshot here)*

2.3 Search Products Based on Availability

Products currently in stock

```
SELECT Product_ID, Product_Name, Stock_Quantity
FROM Product
WHERE Stock_Quantity > 0;
```

This displays products that are available for purchase.

*Output: (paste SQL*Plus screenshot here)*

Products with low stock (fewer than 10 units)

```
SELECT Product_ID, Product_Name, Stock_Quantity
FROM Product
WHERE Stock_Quantity < 10
ORDER BY Stock_Quantity ASC;
```

This helps sellers identify products that need restocking.

*Output: (paste SQL*Plus screenshot here)*

2.4 Search Products by Name

Products whose name contains "Handmade"

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Product_Name LIKE '%Handmade%';
```

The LIKE operator with % matches any text before or after the word.

*Output: (paste SQL*Plus screenshot here)*

3. Retrieve Customer Information

Display all customers

```
SELECT Customer_ID, Customer_Name, Email, Contact_No
FROM Customer;
```

*Output: (paste SQL*Plus screenshot here)*

Search for a particular customer

```
SELECT *
FROM Customer
WHERE Customer_ID = 101;
```

*Output: (paste SQL*Plus screenshot here)*

4. Apply Filtering Conditions

Multiple conditions can be combined using AND, OR, and NOT.

4.1 AND Condition

Products priced below 2000 in category 2

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price < 2000
AND Category_ID = 2;
```

This displays products that satisfy both conditions.

*Output: (paste SQL*Plus screenshot here)*

4.2 OR Condition

Products in category 1 or category 3

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Category_ID = 1
OR Category_ID = 3;
```

This displays products belonging to either category 1 or category 3.

*Output: (paste SQL*Plus screenshot here)*

4.3 NOT Condition

Products not in category 1

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE NOT Category_ID = 1;
```

This displays all products except those in category 1.

*Output: (paste SQL*Plus screenshot here)*

4.4 IN Operator

Products in categories 1, 2 or 3

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Category_ID IN (1, 2, 3);
```

*Output: (paste SQL*Plus screenshot here)*

4.5 BETWEEN Operator

Products priced between 100 and 300

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price BETWEEN 100 AND 300;
```

*Output: (paste SQL*Plus screenshot here)*

5. Basic Business Reports

5.1 Product Price Report

Products ranked by price

```
SELECT
    Product_ID,
    Product_Name,
    Price
FROM Product
ORDER BY Price DESC;
```

*Output: (paste SQL*Plus screenshot here)*

5.2 Customer Order Report

Orders placed by each customer, latest first

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    o.Total_Amount
FROM Customer c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
ORDER BY o.Order_Date DESC;
```

*Output: (paste SQL*Plus screenshot here)*

5.3 Product Order Report

Order items ranked by subtotal

```
SELECT
    oi.Order_ID,
    oi.Product_ID,
    oi.Quantity,
    oi.Unit_Price,
    oi.Subtotal
FROM Order_Item oi
ORDER BY oi.Subtotal DESC;
```

*Output: (paste SQL*Plus screenshot here)*

5.4 Customer-wise Total Orders

Number of orders per customer

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    COUNT(o.Order_ID) AS Total_Orders
FROM Customer c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Orders DESC;
```

*Output: (paste SQL*Plus screenshot here)*

5.5 Customer-wise Total Purchase Amount

Total amount spent per customer

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(o.Total_Amount) AS Total_Purchase
FROM Customer c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Purchase DESC;
```

*Output: (paste SQL*Plus screenshot here)*

Conclusion

In Week 7, basic SQL queries were implemented on the Etsy Marketplace Database System using SELECT, WHERE, ORDER BY, DISTINCT, AND, OR, NOT, IN, BETWEEN and LIKE. Products were searched by price, category and availability, customer details were retrieved, and basic business reports were generated for products, customers and orders.